


**KINGDOM OF CAMBODIA**

**NATION RELIGION KING**

AIIS
Department of Applied Mathematics and Statistics

# Report of Object-Oriented Programming

## Basic of OOP


| Name of Student | ID | Class |
|---|---|---|
| THAT SOKMENGLY | e20240602 | I3_AMS_TP_D |

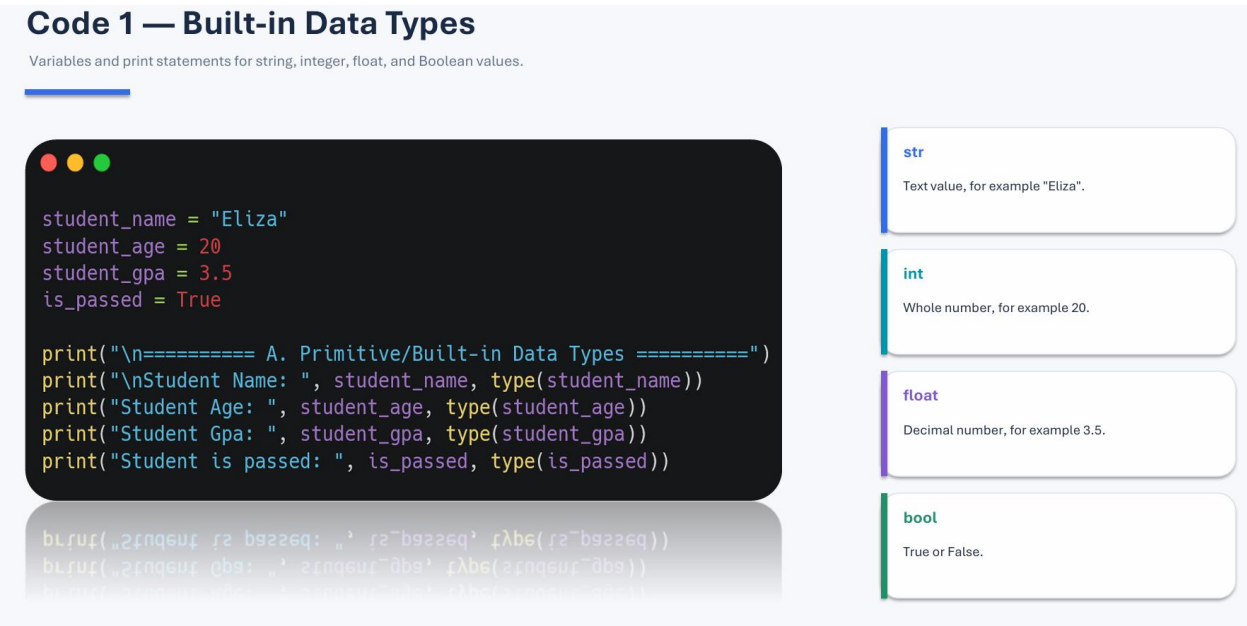
Lecture: Mr. LONG PAKRIGNA (Course)

Mr. VANN VISAL (TP)

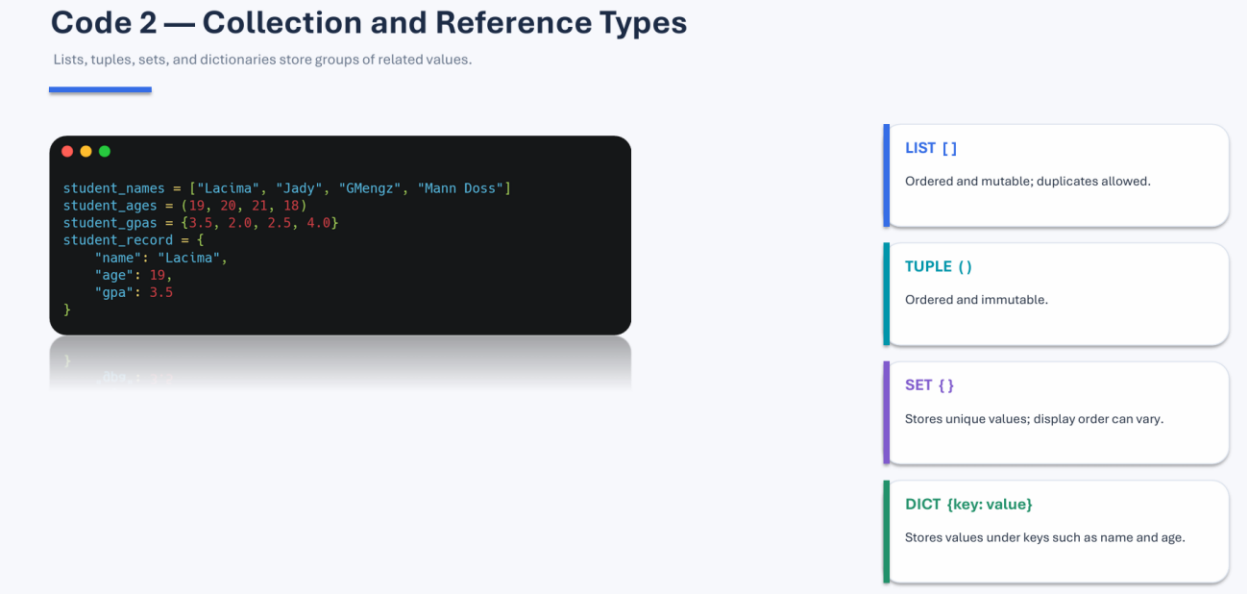

Academic year 2026-2027

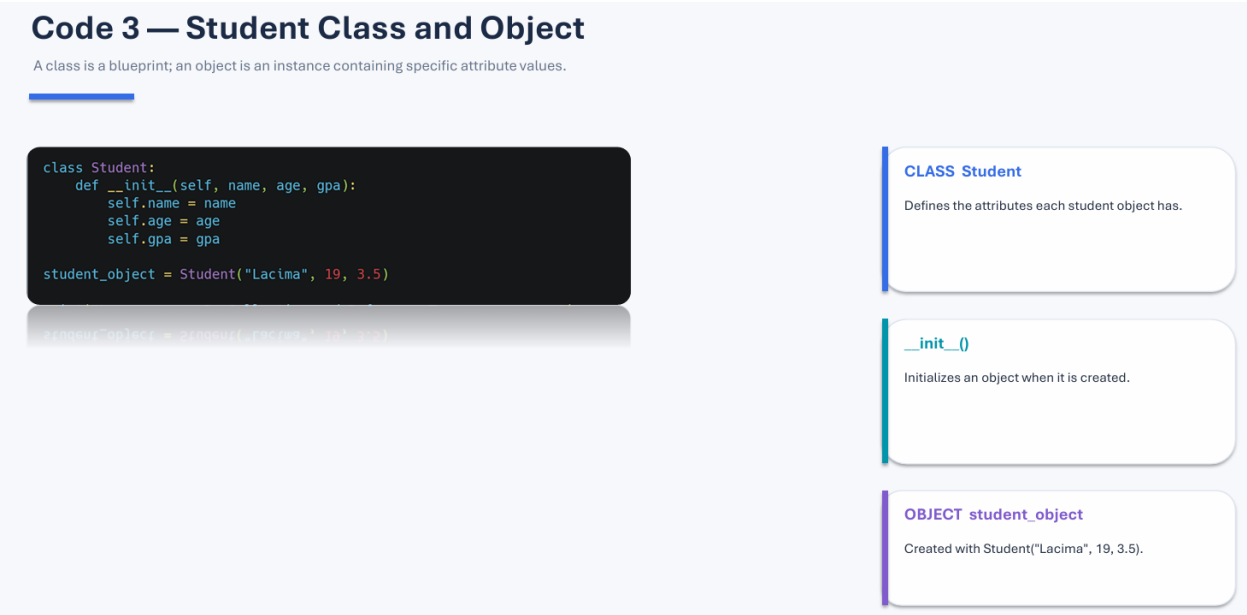
## Topic 1: Python Basic Data Types




**Built-in data types** are used to store different kinds of values, such as strings (`str`) for store as the letter or something, integers (`int`) for store whole number, floating-point numbers (`float`), and Boolean values (`bool`) as condition True or False.




**Collection and reference types** help organize related data. Python collections include lists, tuples, sets, and dictionaries. A list is ordered and changeable, a tuple is ordered and unchangeable, a set stores unique values, and a dictionary stores data in key-value pairs. The presentation also demonstrates how to create a `Student` class and an object using the `__init__()` method and attributes such as name, age, and GPA.

## Code 4 — Collection and Object Output

Print each collection and inspect the Student object and its attributes.

```
print("\n========== B. Collection and Reference Types ==========")
print("\nStudent Names: ", student_names, type(student_names))
print("Student Ages: ", student_ages, type(student_ages))
print("Student Gpas: ", student_gpas, type(student_gpas))
print("Student Record: ", student_record, type(student_record))

print("\n***** Object Output *****")
print("\nObject Type: ", student_object, type(student_object))
print("Object Name attribute: ", student_object.name,
type(student_object.name))
print("Object Age attribute: ", student_object.age,
type(student_object.age))
print("Object Gpa attribute: ", student_object.gpa,
type(student_object.gpa))
```

```
========== A. Primitive/Built-in Data Types ==========

Student Name:  Eliza <class 'str'>
Student Age:  20 <class 'int'>
Student Gpa:  3.5 <class 'float'>
Student is passed:  True <class 'bool'>

========== B. Collection and Reference Types ==========

Student Names:  ['Lacima', 'Jady', 'GMengz', 'Mann Doss'] <class 'list'>
Student Ages:  (19, 20, 21, 18) <class 'tuple'>
Student Gpas:  {3.5, 2.5, 2.0, 4.0} <class 'set'>
Student Record:  {'name': 'Lacima', 'age': 19, 'gpa': 3.5} <class 'dict'>

***** Object Output *****

Object Type:  <__main__.Student object at 0x000001E04F586A50> <class '__main__.Student'>
Object Name attribute:  Lacima <class 'str'>
Object Age attribute:  19 <class 'int'>
Object Gpa attribute:  3.5 <class 'float'>
```

## Code 5 — Python Literals

Literals are values written directly in source code: numbers, text, booleans, None, and collections.

```
num_literal = 19
num_float_literal = 3.5
str_literal = "Lacima"
bool_literal = True
none_literal = None

list_literal = ["Lacima", "Jady"]
tuple_literal = (19, 20)
set_literal = {3.5, 2.0}
dict_literal = {"name": "Lacima", "age": 19}
```

**NUMERIC**

19 is an int; 3.5 is a float.

**STRING / BOOLEAN**

"Lacima" is text; True is Boolean.

**NONE**

None represents no value / not set.

**COLLECTIONS**

List, tuple, set, and dict literals.

```
print("\n========== C. Literals in Python ==========")
print("\nNumeric Integer Literal: ", num_literal, type(num_literal))
print("Numeric Float Literal: ", num_float_literal,
type(num_float_literal))
print("String Literal: ", str_literal, type(str_literal))
print("Boolean Literal: ", bool_literal, type(bool_literal))
print("None Literal: ", none_literal, type(none_literal))
print("Collection Literal (List): ", list_literal, type(list_literal))
print("Collection Literal (Tuple): ", tuple_literal,
type(tuple_literal))
print("Collection Literal (Set): ", set_literal, type(set_literal))
print("Collection Literal (Dict): ", dict_literal, type(dict_literal))
print("\n")
```

```
========== C. Literals in Python ==========

Numeric Integer Literal:  19 <class 'int'>
Numeric Float Literal:  3.5 <class 'float'>
String Literal:  Lacima <class 'str'>
Boolean Literal:  True <class 'bool'>
None Literal:  None <class 'NoneType'>
Collection Literal (List):  ['Lacima', 'Jady'] <class 'list'>
Collection Literal (Tuple):  (19, 20) <class 'tuple'>
Collection Literal (Set):  {2.0, 3.5} <class 'set'>
Collection Literal (Dict):  {'name': 'Lacima', 'age': 19} <class 'dict'>
```

**Python literals** are values written directly in the code, including numbers, strings, Boolean values, None, and collection values. The type( ) function is used to identify the data type of a value.

In conclusion, understanding Python basic data types, collections, classes, objects, and literals is essential for writing and understanding Python programs. These concepts provide a foundation for learning Object-Oriented Programming (OOP) and developing more complex applications.

# Topic 2: **Python Variables and Attributes**

### Local Variables

**Definition ( What is a Local Variable? )**
A temporary variable created inside a function or method. It exists only while that function executes and cannot be accessed outside its scope.

```
class Student:
    def introduce(self): # Local Variable: created inside method
        message= "My name is Jaehyun."
        print(message)

student1 = Student()
student1.introduce()
# Accessing 'message' here raises an error # print(message) #
NameError: name 'message' is not defined
```

**Function Scope**
Created on execution, destroyed as soon as the method finishes.

**NameError Outside Scope**
Hidden from outside code to keep variables isolated and safe.

**Local variables** are variables created inside a function or method. It is generally accessible only within that function or method.

### Instance Attributes

**Definition ( What is an Instance Attribute? )**
Variables attached to individual objects via self inside __init__(). Each object created from the class maintains its own unique state.

```
class Student:
    def __init__(self, name, grade):
        # Instance attributes (Unique per object)
        self.name = name
        self.grade = grade

# Creating two distinct instances (objects)
student1 = Student("Sokha", 10)
student2 = Student("Vicheka", 11)

print(student1.name, student1.grade) # Output: Sokha 10
print(student2.name, student2.grade) # Output: Vicheka 11
```

**Constructor (`__init__`)**
Runs automatically to set up initial data whenever an object is created.

**Independent Object Memory**
Modifying `student1` data never alters `student2` data.

**instance attributes** are variables that belong to a specific object. They are commonly defined using `self` inside the `__init__()` method. Each object can have different attribute values.

### Complete Practical Example

```python
class Student:
    # PART C: Class Attribute (Shared blueprint data)
    school_name = "ITC"

    def __init__(self, name, grade):
        # PART B: Instance Attributes (Unique object data)
        self.name = name
        self.grade = grade

    def introduce(self):
        # PART A: Local Variable (Temporary scope)
        msg = f"Hi, I'm {self.name} (Grade {self.grade}) at
{Student.school_name}."
        print(msg)
# Demonstration
s1 = Student("Sokha", 10)
s2 = Student("Vicheka", 11)
 s1.introduce()
s2.introduce()
```

**Part A: Local Variable (`msg`)**
Temporary variable exists only inside `introduce()` function.

**Part B: Instance Attributes (`self.name`, `self.grade`)**
Unique data for `s1` ("Sokha") and `s2` ("Vicheka").

**Part C: Class Attribute (`school_name`)**
Shared school name ("ITC") across all student objects.

### Class Attributes

**Definition ( What is a Class Attribute? )**
Variables defined directly inside the class block (outside methods). They belong to the blueprint itself and are shared equally by all instances.

```python
class Student:
      # Class attribute (Shared by ALL instances)
      school_name = "Institute of Technology of Cambodia"

student1 = Student()
student2 = Student()
print(student1.school_name) # Output: Institute of Tech...
print(student2.school_name) # Output: Institute of Tech...
 # Updating at class level updates it everywhere
Student.school_name = "ITC"
print(student1.school_name)
 # Output: ITC
```

**Class-Level Declaration**
Declared directly in class scope without using `self.`.

**Shared Memory Space**
All objects read the exact same value from the class blueprint.

**Class attributes** are variables defined inside a class but outside its methods. They are shared by instances of the class unless an instance overrides the attribute with its own value.

In conclusion, local variables store temporary data within functions, instance attributes store data specific to individual objects, and class attributes store data shared across objects. Understanding these differences is essential for writing organized and effective Python programs using OOP.

# Topic 3: Python Modifier and Access Control

# Public Naming Convention

01

Public naming convention can access from anywhere inside or outside the class.It usually no prefix(self.name)

02

```python
class Product:
    def __init__(self):
        self.price=15

item=Product()
print(item.price)

```

# Non-Public Naming Convention

01 Meant for internal use within the class and its subclasses (though not strictly enforced).

02 Single leading underscore _ (e.g., self._name)

```python
class Product:
    def __init__(self):
        self.price = 10.00
        self._stock = 50
item = Product()
print(item.price)
print(item._stock)
```

# Private Naming Convention

## MEANING

Private naming convention is a set of rules for naming variables and methods intended for internal use rather than public access.

```python
class Student:
    def __init__(self):
        self._grade = "A"
        self.__grade = "A"

    def show_grade(self):
        return self.__grade

student = Student()
print("Private Grade:", student.show_grade()
```

## Access Control by Convenntion and Name Mangling

# Meaning

Access Control by Convention means using naming rules to indicate how attributes and methods should be accessed in a Python class.

Name Mangling is a Python mechanism that changes the internal name of an attribute or method that starts with two underscores (__), subject to certain naming rule

# Access Control by Convenntion

```python
class Student:
    def __init__(self):
        self.name = "Vathana"
        self._age = 20
student = Student()

print(student.name)
print(student._age)
```

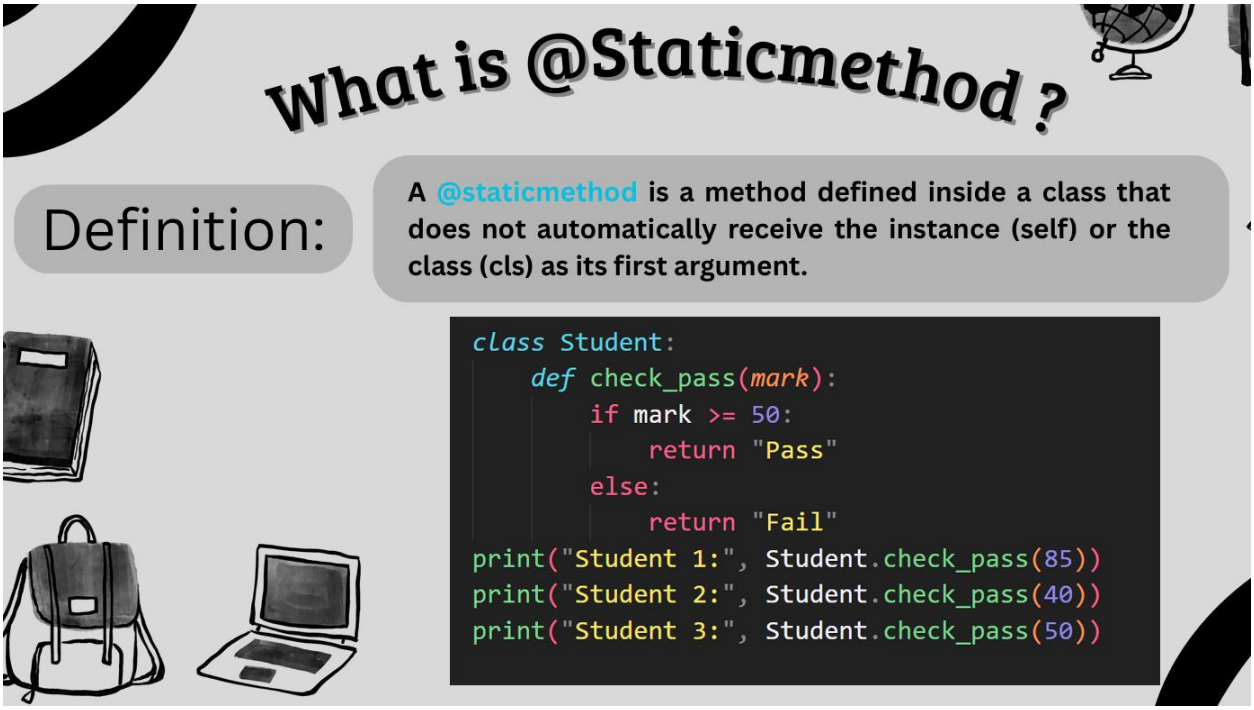
# Name Mangling

```python
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance

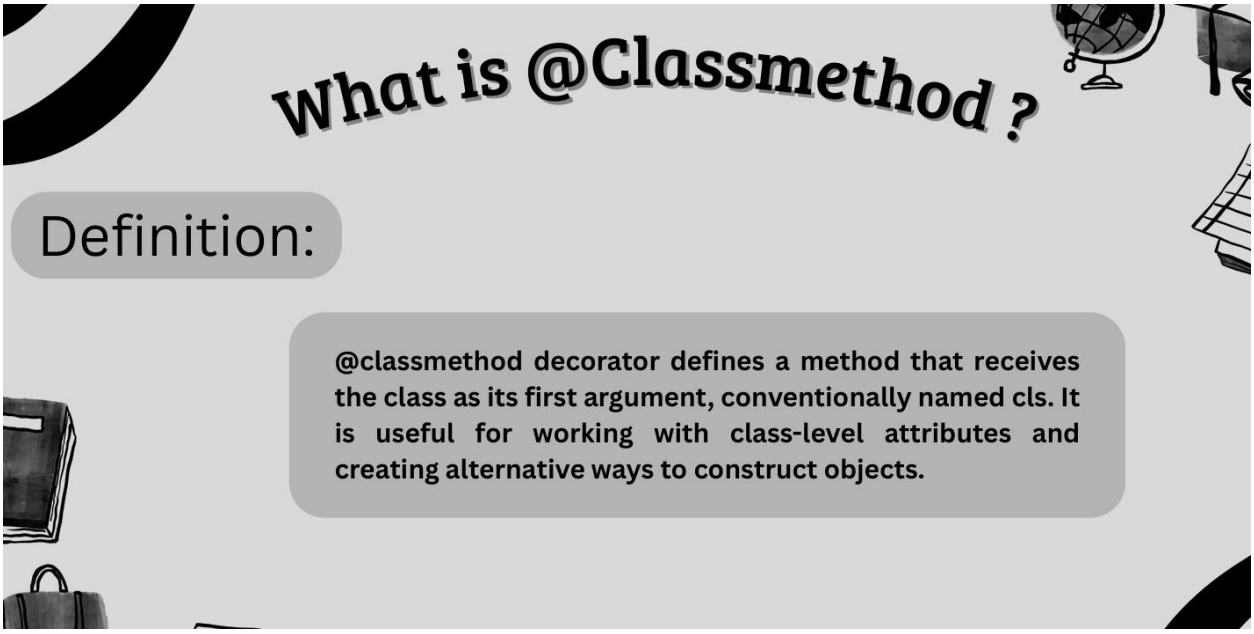
    def show_balance(self):
        return self.__balance


account = BankAccount(1000)

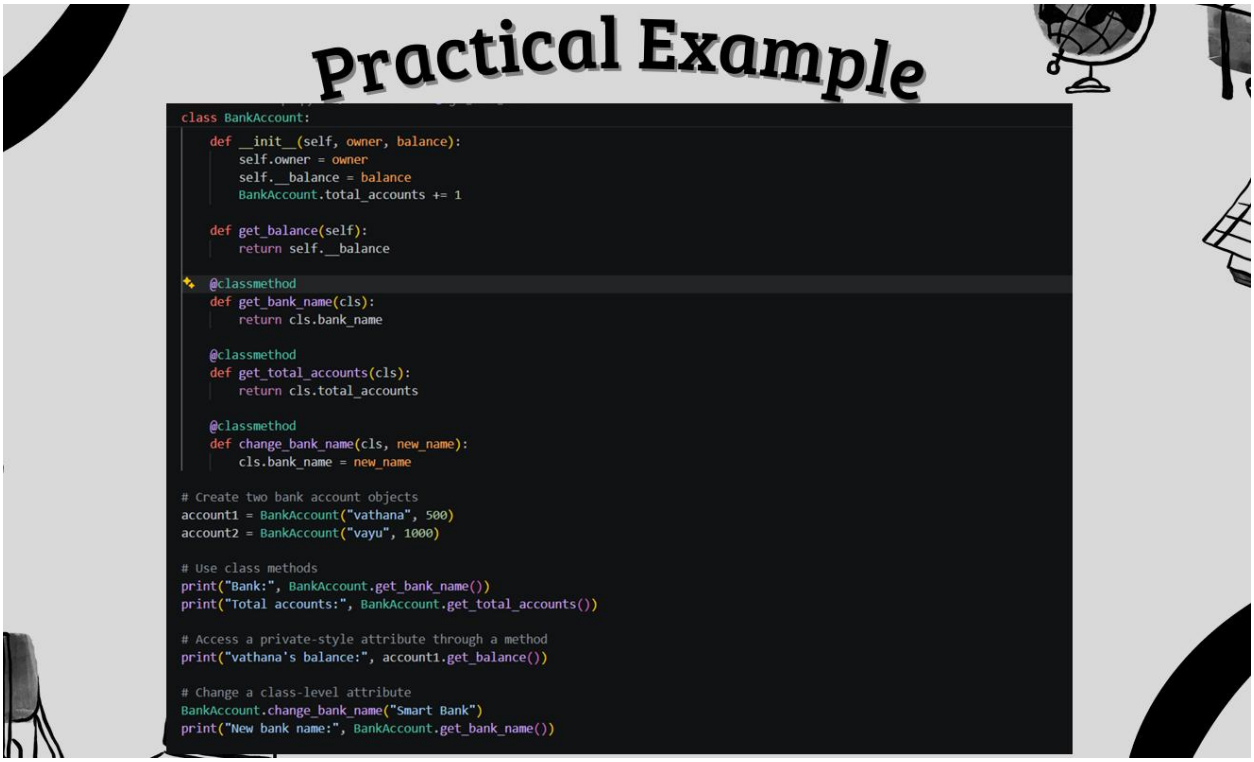
print(account.show_balance())
print(account._BankAccount__balance)
```




The `@staticmethod` **decorator** defines a method that does not automatically receive `self` or `cls` as its first argument. It is useful for functions related to a class that do not need instance or class data.




The `@classmethod` **decorator** defines a method that receives the class as its first argument, conventionally named `cls`. It is useful for working with class attributes and creating alternative ways to construct objects.




The practical example demonstrates how these concepts can be applied to a **Bank Account** class, including managing account information, accessing attributes, counting accounts, and changing class-level data.

In conclusion, understanding Python modifiers and access control helps developers write organized, maintain, and reusable code. These concepts are important for protecting internal implementation details, managing class and instance data, and applying OOP principles effectively.

## Topic 4: Python Modifiers and OOP Features-part 2

# Abstract classes and methods



## what is an Abstract classes?

An Abstract Classes is a class that provides a general structure for other classes. It cannot be instantiated directly when it contains methods that have not been implemented.

**Example:** A Shape class can be an abstract class because different shapes, such as circles and rectangles, calculate their areas differently.

## what is an Abstract methods?

An abstract method is a method declared in an abstract class without providing its complete implementation.

- ABC stands for Abstract Base Class. It is a class provided by Python's abc module.
- @abstractmethod decorator marks a method that must be implemented by a concrete subclass before its objects can be created.

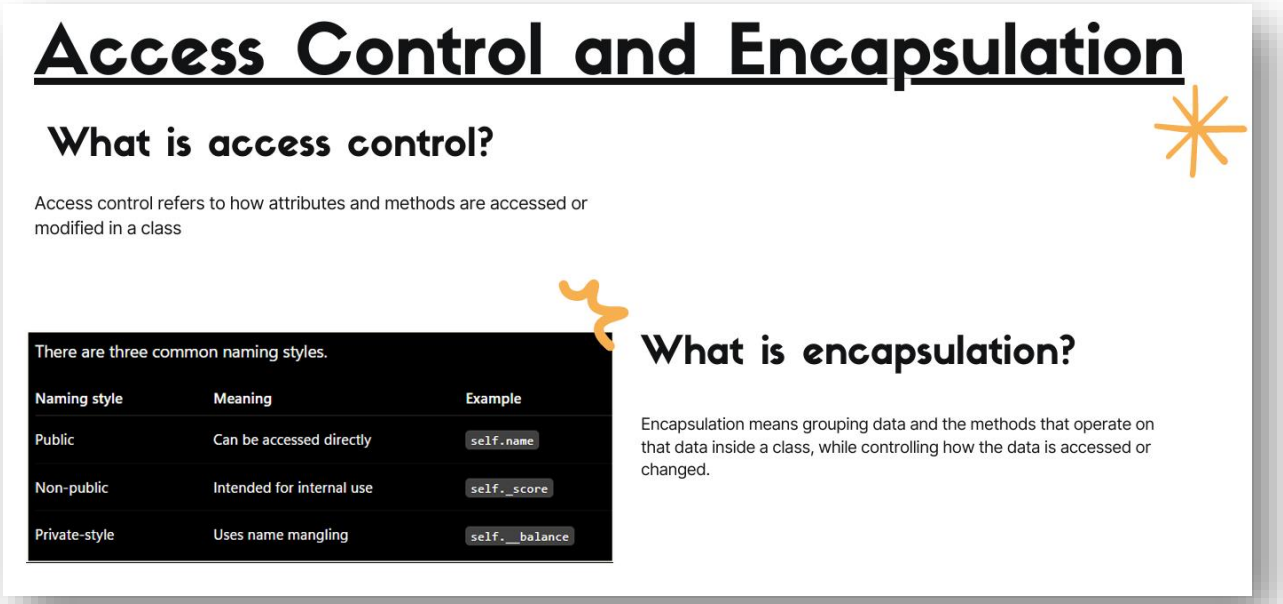
## Program:

```
1
2  from abc import ABC, abstractmethod
3
4  class Shape(ABC):
5      @abstractmethod
6      def area(self):
7          pass
8
9  class Circle(Shape):
10     def area(self):
11         return 3.14 * 5 * 5
12
13 class Rectangle(Shape):
14     def area(self):
15         return 4 * 6
16
17 c = Circle()
18 r = Rectangle()
19
20 print("Circle area:", c.area())
21 print("Rectangle area:", r.area())
```

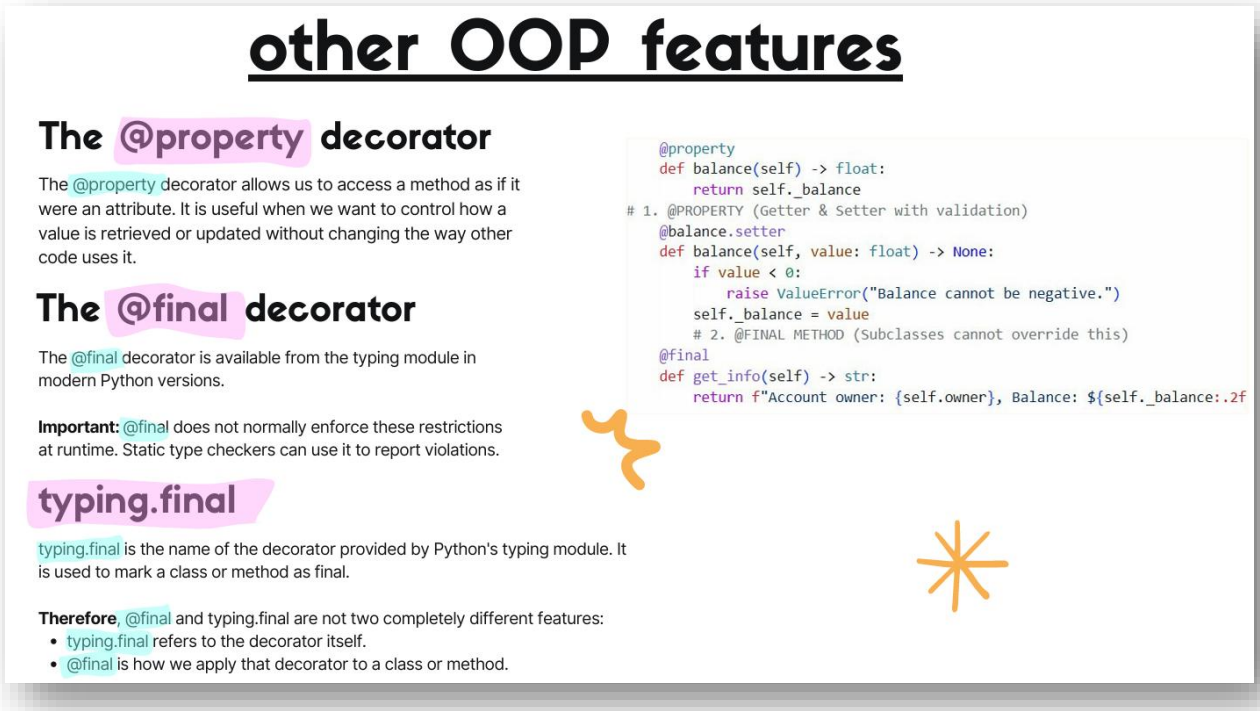
## Output:

```
PS C:\Users\thorn\AppData\Local\
Circle area: 78.5
Rectangle area: 24
PS C:\Users\thorn\AppData\Local\
```

**Abstract classes and methods** provide a general structure for other classes. An abstract class cannot be instantiated directly when it contains unimplemented abstract methods. The `ABC` class and `@abstractmethod` decorator are used to define methods that concrete subclasses must implement.



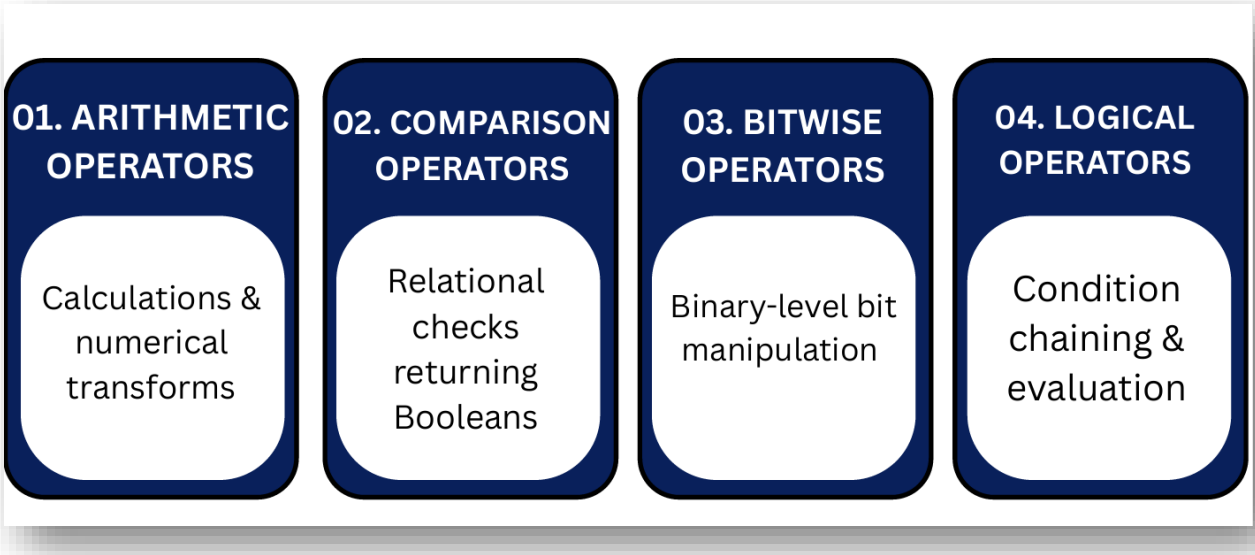

**Access control and encapsulation** describe how attributes and methods are accessed or modified. Encapsulation groups data and the methods that operate on it within a class while controlling how the data is accessed or changed.




The `@property` **decorator** it allows a method to be accessed like an attribute and helps control how values are retrieved or updated and the `@final` **decorator**, provided by the `typing` module, which marks a class or method as final so that static type checkers can identify certain attempts to override or extend it.

In conclusion, abstract classes, encapsulation, access control, properties, and final decorators help developers design more organized, reusable, and maintainable Python programs.

# Topic 5: Python Operators




## Arithmetic Operators

Perform standard mathematical calculations in Python expressions.

| Op | Name | Example | Result |
|---|---|---|---|
| + | Addition | 5 + 3 | 8 |
| - | Subtraction | 5 - 3 | 2 |
| * | Multiplication | 5 * 3 | 15 |
| / | Division | 10 / 3 | 3.333... |
| // | Floor Division | 10 // 3 | 3 |
| % | Modulo | 10 % 3 | 1 |
| ** | Exponentiation | 2 ** 3 | 8 |

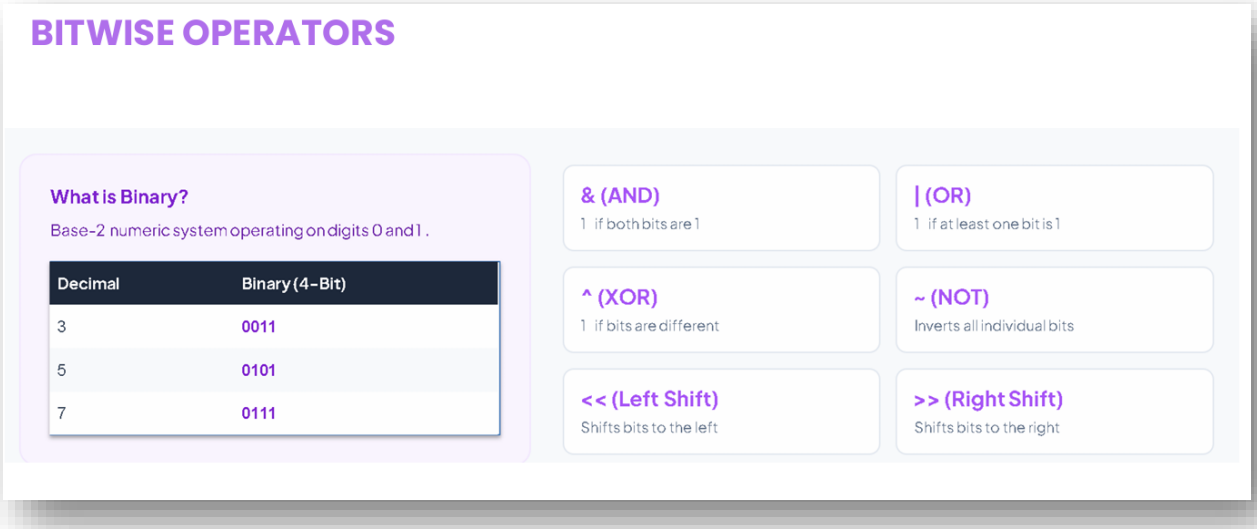
```
print(10 / 2) # 5.0 (Always float)
print(10 // 3) # 3 (Rounds down)
print(10 % 3) # 1 (Remainder)
print(2 ** 3) # 8 (Power)
```

Practical Application
Used for calculating student score averages, total percentage marks, and leftover remainders in batching operations

**Arithmetic operators** perform mathematical calculations, including addition (+), subtraction (-), multiplication (*), division (/), floor division (//), modulo (%), and exponentiation (**). They are useful for calculating totals, averages, percentages, and remainders.




**Comparison operators** compare two values and return a Boolean result, either `True` or `False`. These operators include equal to (==), not equal to (`!=`), greater than (>), less than (<), greater than or equal to (>=), and less than or equal to (<=). It is important to distinguish = for assignment from == for equality checking.




**Bitwise operators** manipulate the individual bits of integers using binary representation. The main operators are AND (`&`), OR (`|`), XOR (`^`), NOT (`~`), left shift (<<), and right shift (>>). Left and right shifts move bits to the left or right, respectively.

# BITWISE OPERATORS

### & (AND)

Both bits must be 1

```
0101 (5)
& 0011 (3)
0001 (1)
```

print(5 & 3) → 1

### | (OR)

At least one bit is 1

```
0101 (5)
| 0011 (3)
0111 (7)
```

print(5 | 3) → 7

### ^ (XOR)

Bits must be different

```
0101 (5)
^ 0011 (3)
0110 (6)
```

print(5 ^ 3) → 6

INVERSION

### ~ Bitwise NOT

Inverts all bits using rule: ~n = -(n+1)

```
print(~5) # -6
```

Flips every 0 to 1 and 1 to 0 in two's complement form.

MULTIPLY X2

### << Left Shift

Moves bits left. Appends 0s on right.

```
print(5 << 1) # 10
0101 → 1010
```

Equivalent to multiplying number by 2n.

DIVIDE /2

### >> Right Shift

Moves bits right. Discards lost bits.

```
print(5 >> 1) # 2
0101 → 0010
```

Equivalent to floor dividing number by 2n.

# Bitwise Shifts (<< and >>)

Move bits left to multiply by two; move them right to floor-divide by two.

```
score = 0b0011  # decimal 3
doubled = score << 1
halved = score >> 1
print(bin(doubled), doubled)  # 0b110, 6
print(bin(halved), halved)    # 0b1, 1
```

LEFT ×2




RIGHT ÷2



## LOGICAL OPERATORS

1. **and:** Returns True if both conditions are True
2. **or:** Returns True if at least one condition is True
3. **not:** Reverses the Boolean value of a condition

| A | B | A and B | A or B |
|---|---|---|---|
| True | True | True | True |
| True | False | False | True |
| False | True | False | True |
| False | False | False | False |

+Code:

```python
score = 80
attendance = 85

# 1. AND: Both conditions must be True
print(score >= 50 and attendance >= 75)
# Output: True

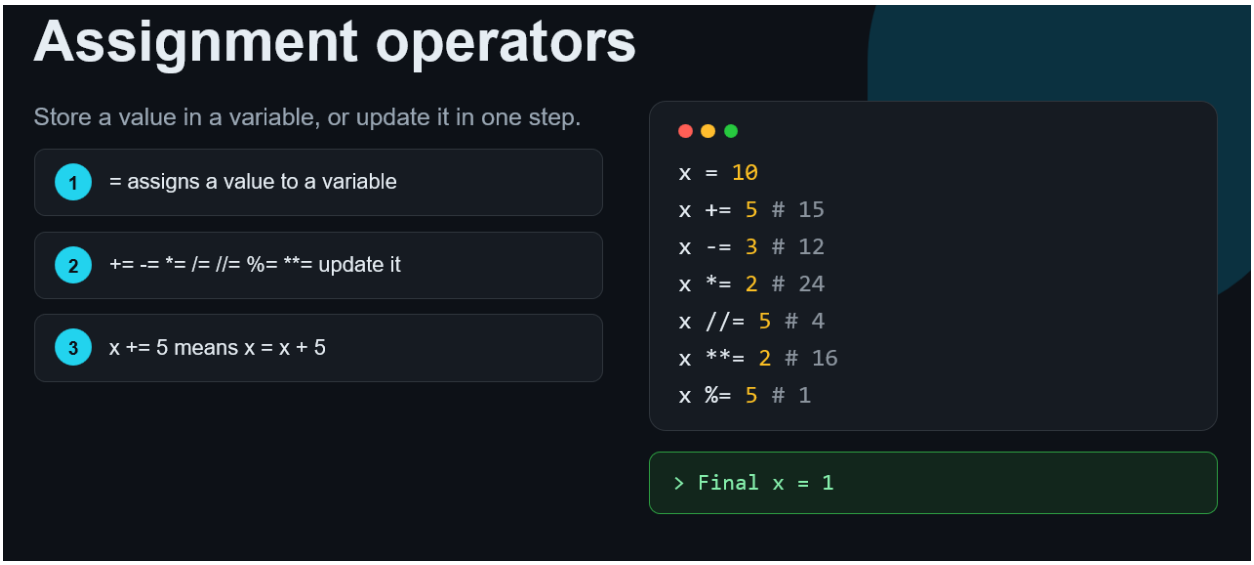
# 2. OR: At least one condition must be True
print(score >= 50 or attendance >= 90)
# Output: True

# 3. NOT: Reverses the Boolean result
print(not score >= 50)
# Output: False
```

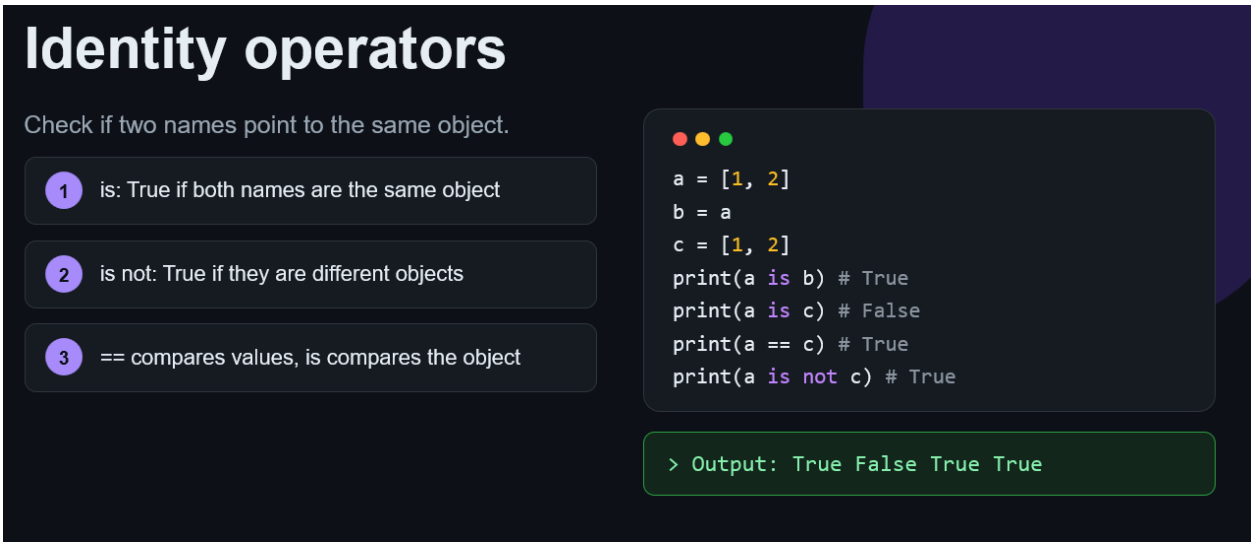
**Logical operators** combine or reverse conditions. The `and` operator returns `True` when both conditions are true, `or` returns `True` when at least one condition is true, and `not` reverses a Boolean value. These operators are useful for checking multiple conditions in a program.

In conclusion, Python **operators** are essential for performing calculations, comparing values, manipulating binary data, and evaluating conditions. Understanding how these operators work helps programmers develop logical thinking and write effective Python programs.

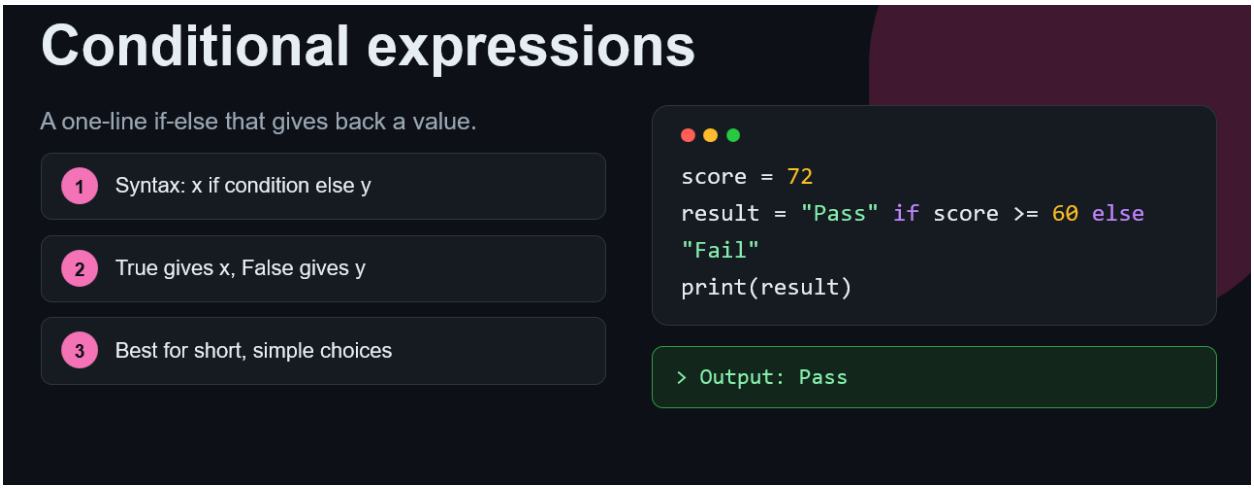
# Topic 6: Python Operators (part 2)



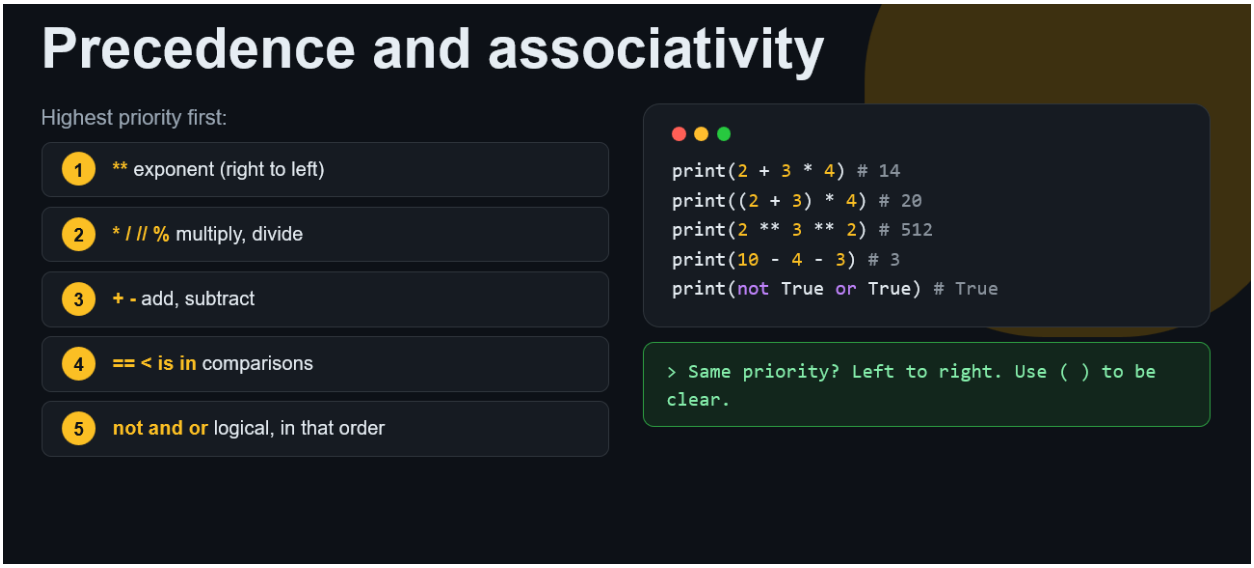

**Assignment operators** are used to assign values to variables and update them efficiently. The basic assignment operator is `=`, while compound assignment operators include `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, and `**=`. For example, `x += 5` is equivalent to `x = x + 5`.



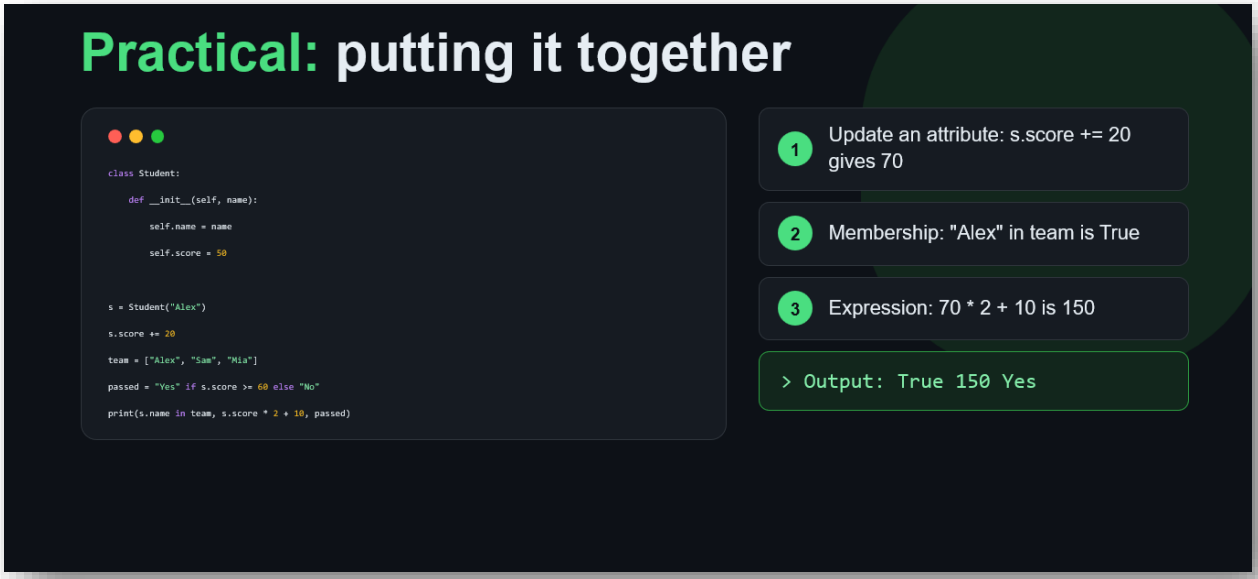

**Identity operators** (`is` and `is not`) check whether two variables refer to the same object in memory. Unlike identity operators, the equality operator (`==`) compares values. **Membership operators** (`in` and `not in`) check whether a value exists in a collection, such as a list or string.

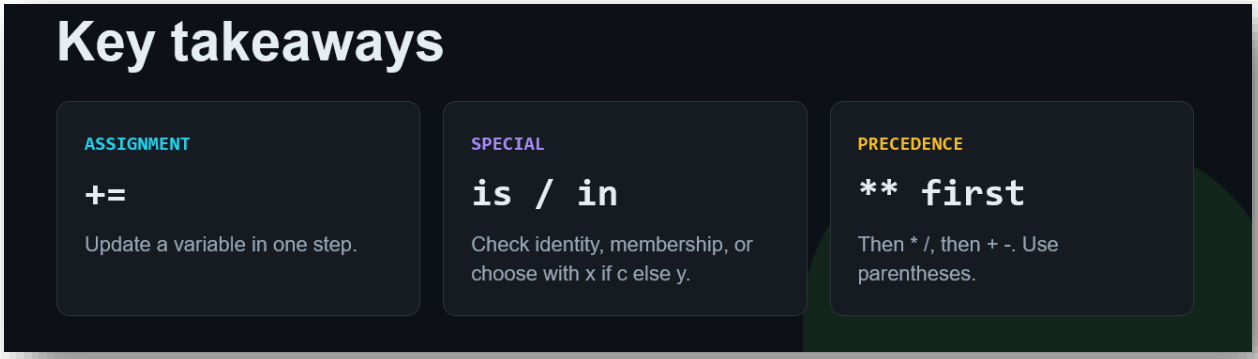



**Conditional expressions** provide a short way to write an `if-else` statement in one line using the syntax `value_if_true if condition else value_if_false`. They are useful for simple decisions, such as determining whether a student passes or fails based on their score.



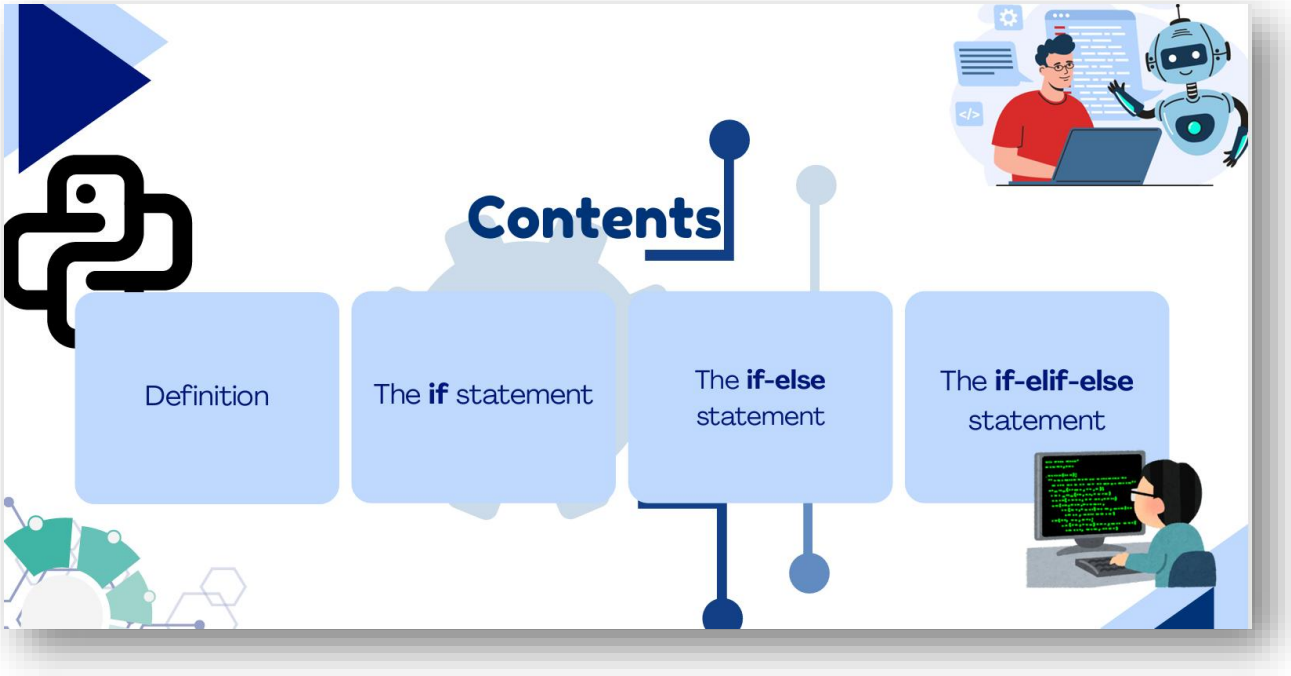

**Operator Precedence and Associativity** determine the order in which Python evaluates expressions. Exponentiation (`**`) is evaluated before multiplication and division, which take priority over addition and subtraction. Parentheses can change the evaluation order and make expressions clearer.

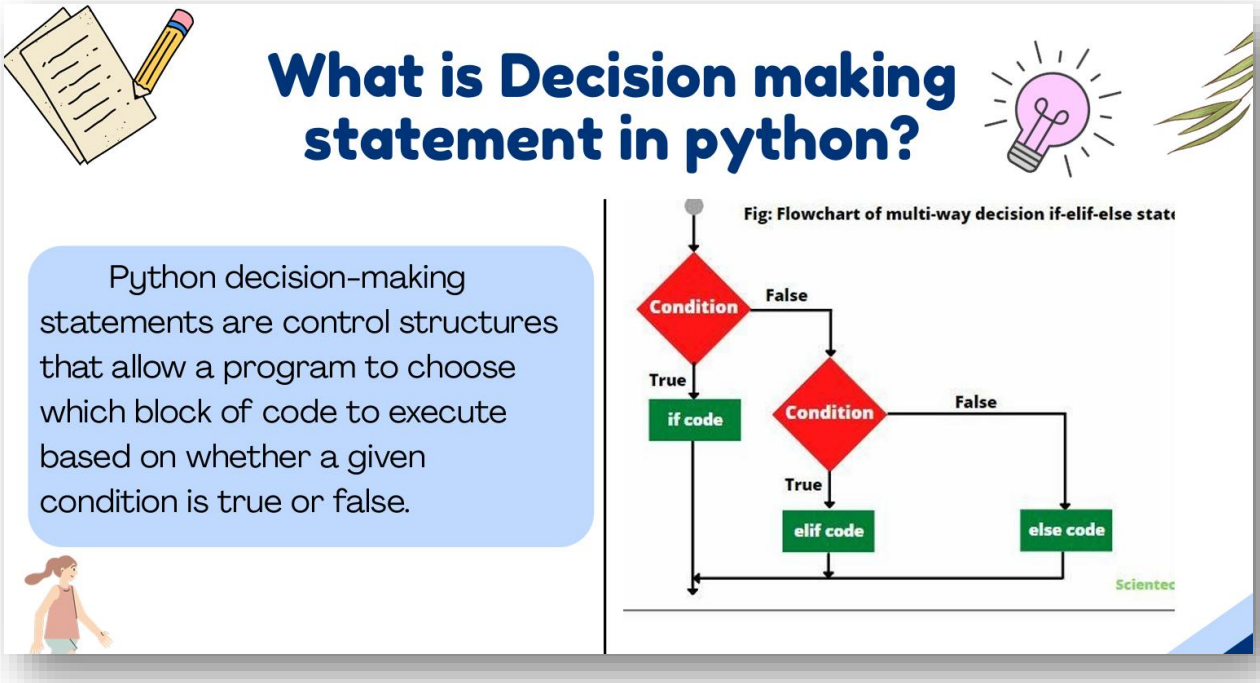



In conclusion, understanding Assignment operators, identity and membership operators, conditional expressions, and operator precedence helps programmers write clearer, more efficient, and more accurate Python programs. Using parentheses when necessary is an important practice for avoiding confusion in complex expressions.

# Topic 7: PYTHON DECISION MAKING STATEMENT



# The If Statement

Evaluates a single condition. If the condition is True, the indented code block executes; if False, it is skipped.

```
score = 86
if score >= 50:
  print("You passsed the exam.")
```

Output :

```
You passsed the exam.
```

**If statement** evaluates a single condition. If the condition is `True`, the indented code block executes; otherwise, it is skipped.

# The if-else statement

## Definition

Allows for multiple mutually exclusive conditions; you can have zero or more elif clauses

```
score = 40
if score >= 50:
  print("You passed the exam.")
else:
  print("You failed the exam.")
```



**Output:**

```
You failed the exam.
```

**if-else statement** provides two possible paths. If the condition is **True**, the if block executes; if it is **False**, the else block executes. For example, it can determine whether a student passes or fails an exam.

# The if-elif-else statement

The if-elif-else statement is the decisions base on conditions. It allows the program to choose which block of code to execute.

```
score = 85
if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
elif score >= 50:
    print("Grade D")
else:
    print("Grade F")
```

**Output:** Grade B

**if : Checks the first condition**

**elif : checks another conditions when the previous condition is false**

**else : execute when all previous conditions are false**

**if-elif-else statement** is used to evaluate multiple conditions sequentially. The **if** statement checks the first condition, **elif** checks additional conditions when previous conditions are false, and **else** executes when all previous conditions are false. It is useful for grading systems that assign different grades according to students' scores.



# Conclusion

**Key Takeaways:**

**if Statement**: Evaluates a single condition; executes code only when True.

**if-else Statement:** Provides a binary choice, handling both True and False outcomes.

**if-elif-else Statement:** Manages multiple conditions sequentially for complex branching logic.

**Practical Application:** Essential for real-world automation, such as grading systems, user authentication, and data validation

In conclusion, Python decision-making statements are essential for controlling program flow and making logical decisions. They are widely used in practical applications such as grading systems, user authentication, and data validation.

# Topic 8: PYTHON DECISION MAKING STATEMENT (Part2)

## D. Nested if Statements

03

### Definition

A nested if is an if statement placed inside another if statement.

```
score = 85

if score >= 50:
    if score >= 80:
        print("Excellent")
    else:
        print("Passed")
else:
    print("Failed")
```

### How it works

- First, check whether score ≥ 50.
- If true, check whether score ≥ 80.
- A score of 85 prints “Excellent”.

**Remember: Python uses indentation to show which statements belong to each if.**

**Nested if statements** allow one `if` statement to be placed inside another `if` statement. They are useful when a program needs to check one condition before checking another. For example, a student's score can be checked to determine whether the student passed or achieved an excellent result.

## E. Multiple Branches: if-elif-else

Use if-elif-else when the program must choose between several conditions.

```
score = 75
if score >= 90:
    print("Grade A")
elif score >= 80:
    print("Grade B")
elif score >= 70:
    print("Grade C")
elif score >= 50:
    print("Grade D")
else:
    print("Grade F")
```

### Output

```
PS D:\Python\AMS-I3\OOP> python -
Grade C
PS D:\Python\AMS-I3\OOP>
```

The conditions are checked from top to bottom. The first true branch runs, and the remaining branches are skipped.

**multiple branches using if-elif-else** allow a program to choose between several conditions. Python checks the conditions from top to bottom and executes the first branch whose condition is true. The `match-case` statement provides another way to select an action by matching a value against different cases.

## E. Multiple Branches: match-case

05

match-case compares a value with different patterns. It requires Python 3.10 or newer.

```
choice = "2"

match choice:
    case "1":
        print("Add student")
    case "2":
        print("View students")
    case "3":
        print("Search student")
    case "4":
        print("Exit")
    case _:
        print("Invalid choice")
```

### Key points

- case "1" matches the value "1".
- Each case represents an option.
- case _ is the default case.
- Use if-elif-else for general conditions or ranges.

### Out put:

```
PS D:\Python\AMS-I3\OOP>
View students
```

## F. Conditional Expressions

A short way to choose one of two values based on a condition.

### Java

```
result = condition ? value1 : value2;
```

### Python

```
result = value1 if condition else value2
```

### Example

```
score = 65
result = "Pass" if score >= 50 else "Fail"
print(result)
```

### Output: Pass

If score is at least 50, choose “Pass”; otherwise choose “Fail”.

**conditional expressions** provide a short way to choose between two values based on a condition. They use the syntax `value_if_true if condition else value_if_false` and are suitable for simple decisions, such as determining whether a student passes or fails.

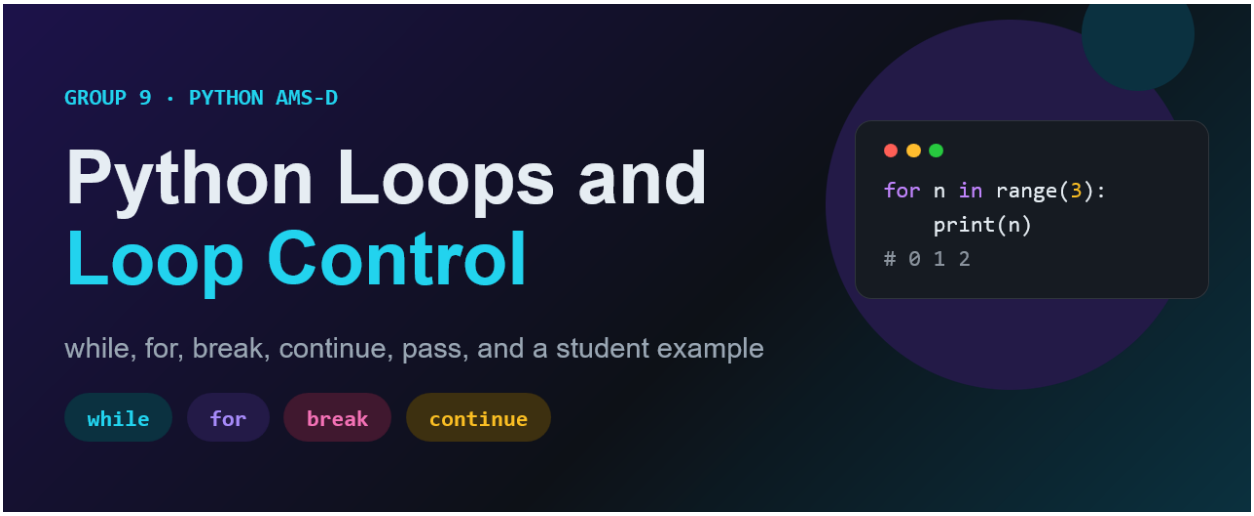
## CONCLUSION

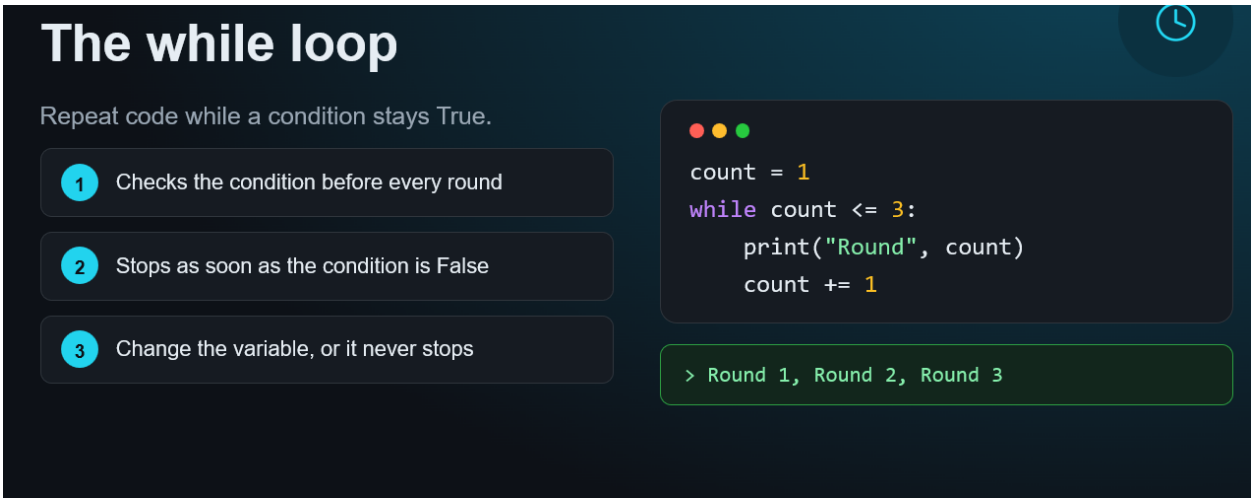
**Python decision-making helps programs choose the correct action based on conditions.**

- Nested if: checks a condition inside another condition.
- if-elif-else / match-case: chooses among multiple alternatives.
- Conditional expression: selects one of two values in one line.
- Student Management System: demonstrates these ideas in practice.

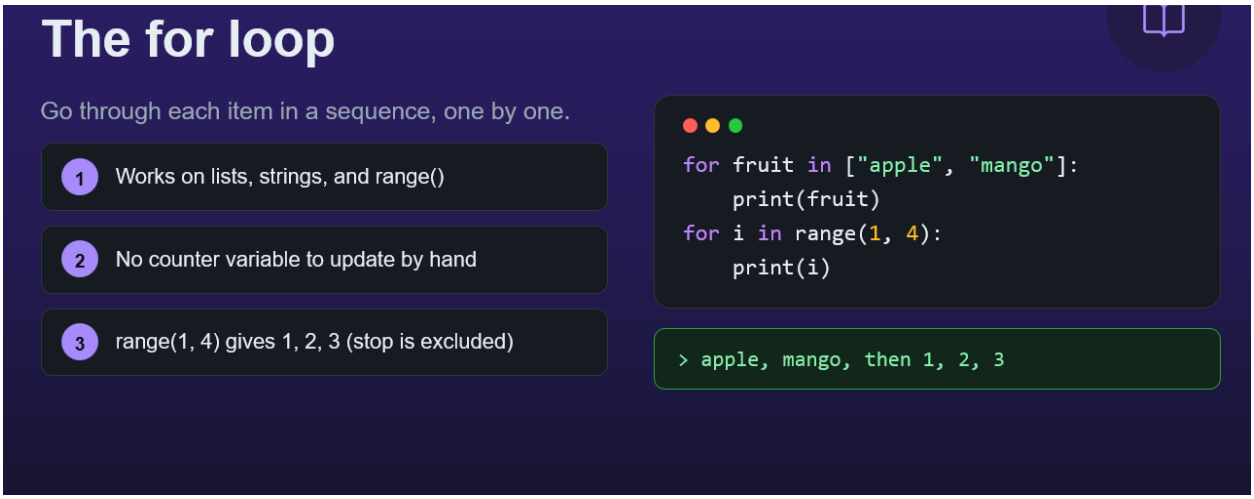
**In conclusion**, Python decision-making techniques help programs evaluate conditions and select appropriate actions. Understanding nested **if** statements, multiple branches, conditional expressions, and their practical applications enables programmers to develop more flexible, organized, and useful Python programs.

# Topic 9: PYTHON LOOPS AND LOOP CONTROL



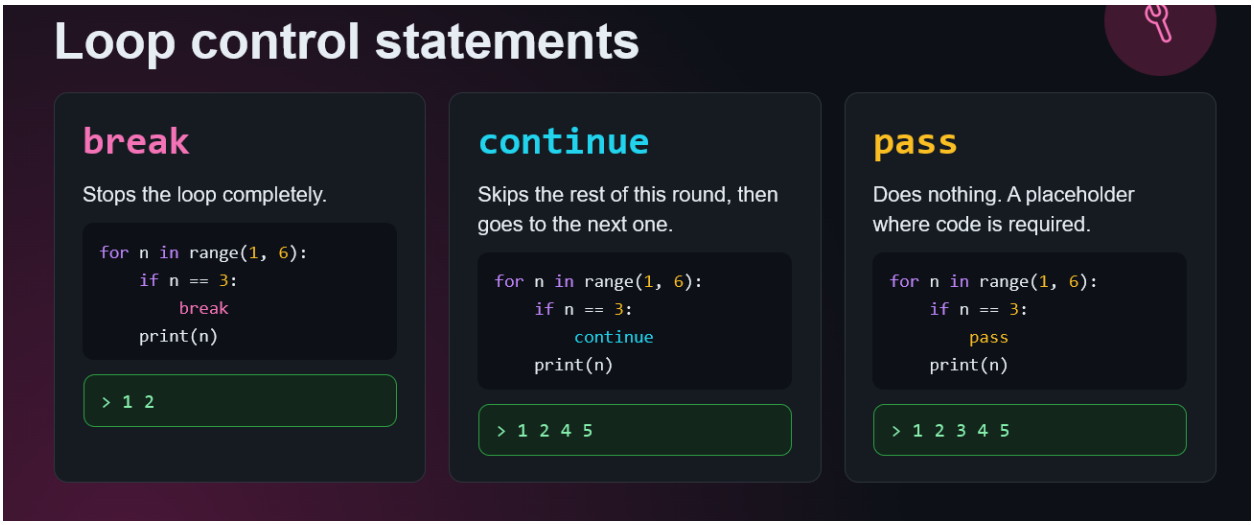



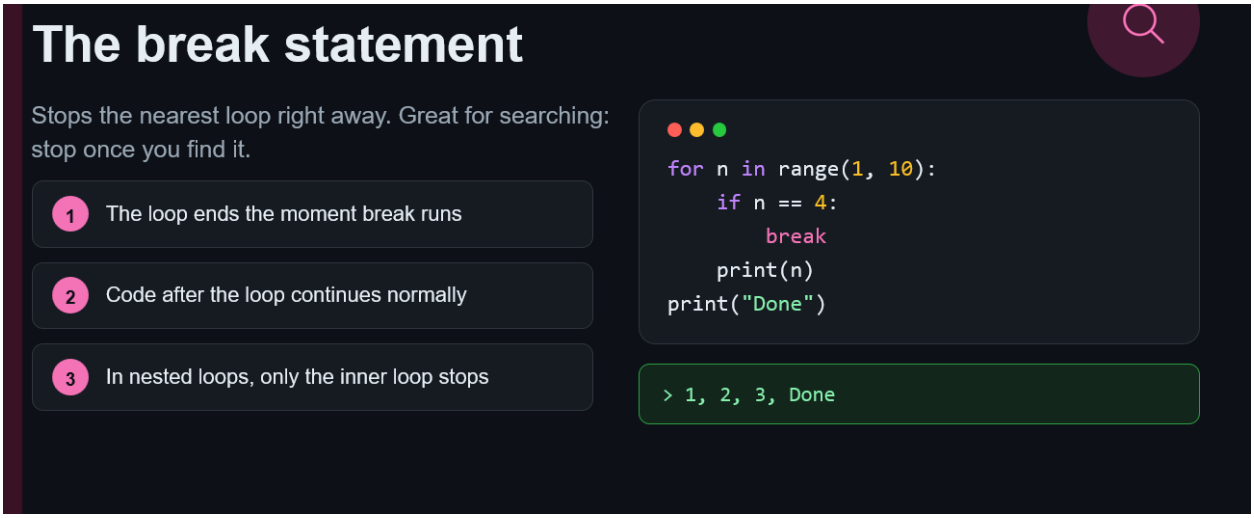
**while loop** repeats a block of code as long as a condition remains True. The condition is checked before each iteration, and the loop stops when the condition becomes False. Programmers must update the relevant variable to avoid an infinite loop.



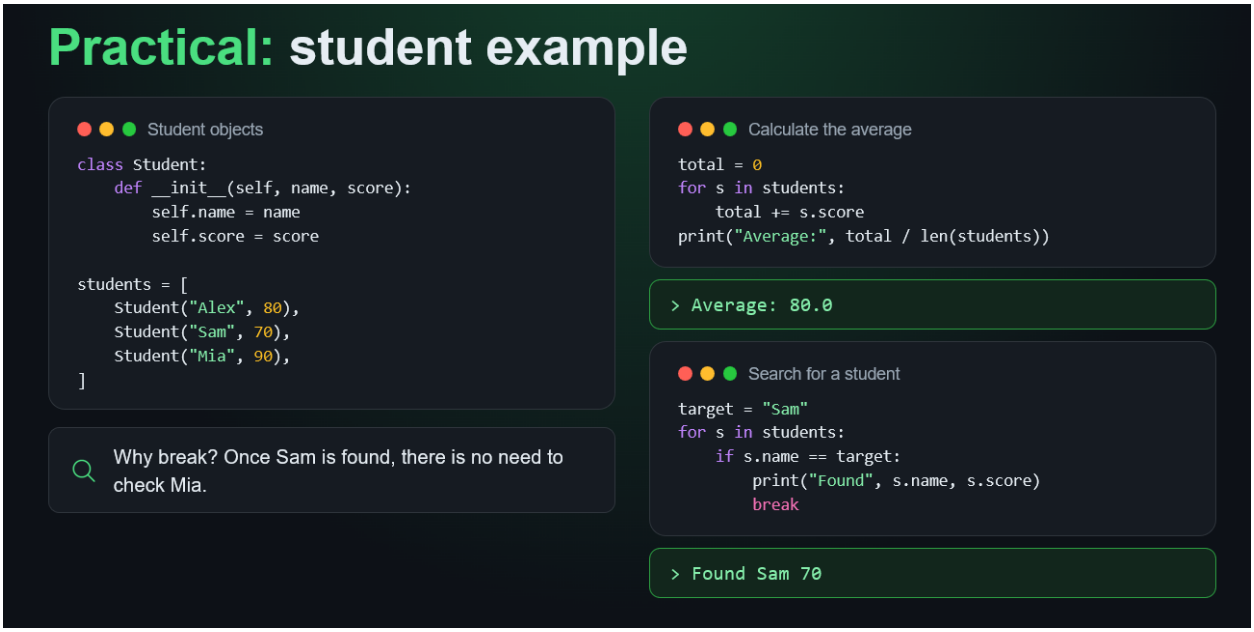

**for loop** iterates through items in a sequence, such as a list, string, or `range()`. It is useful when processing collections or repeating code a specific number of times. The presentation also explains that Python does not have a built-in `do-while` loop, but similar behavior can be achieved using `while True` and `break`.



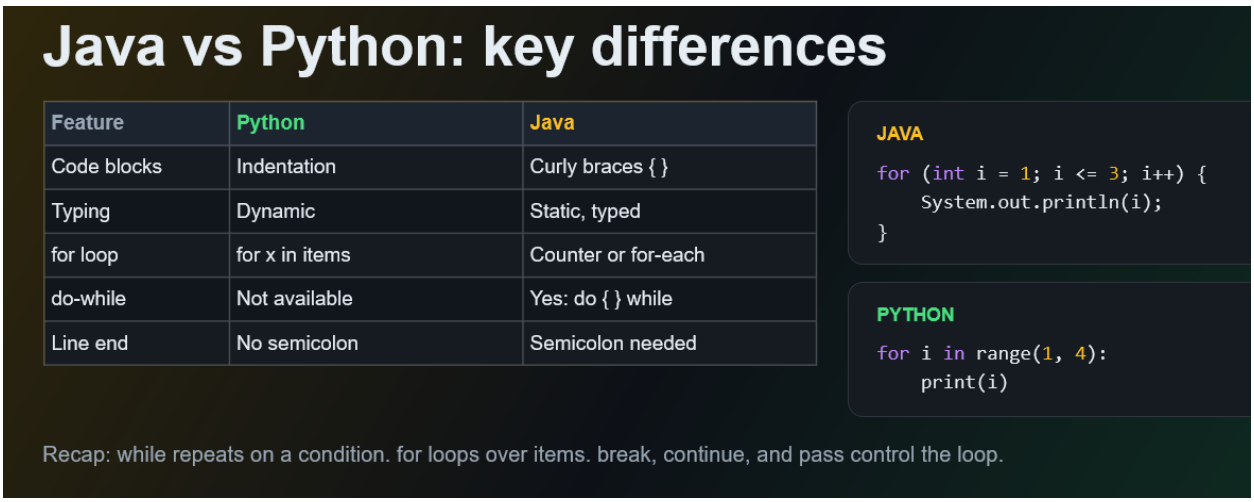



**loop control statements** determine how a loop behaves. The `break` statement immediately stops the nearest loop, `continue` skips the remaining code in the current iteration and moves to the next iteration, and `pass` performs no action and serves as a placeholder.




The **practical example** demonstrates how loops work with Object-Oriented Programming through a `Student` class. A list of student objects is used to calculate the average score and search for a particular student. The `break` statement stops the search once the target student is found.




This is the comparison between **Python and Java**, highlighting differences in indentation, typing systems, loop syntax, do-while support, and semicolon usage.

In conclusion, understanding Python loops and loop control statements helps programmers automate repetitive tasks, process collections of data, and build efficient programs. These concepts are essential for solving programming problems and developing practical applications.

😊 **THANK YOU TEACHER** 😊